

2.2.1 Modularity

ماڈیولریتی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 06 of 24

2.2.1 Modularity

LEARN IT

Modularity means breaking an algorithm into smaller, independent parts such as **functions, classes or modules**. This makes the algorithm more organised and easier to work with.

How to evaluate modularity

1. **Single Responsibility Principle (SRP)**. Each module should focus on **one** specific task. If a function calculates a sum, prints a report and saves a file, it violates SRP.
2. **Coupling**. Modules should be reusable in different contexts. Good modularity means **loose coupling**: a change in one part does not smash the others. Tight coupling is when two functions share so many hidden details that you cannot reuse one without the other.
3. **Abstraction**. A well-designed algorithm hides complex implementation details. You can call a sorting function without knowing whether it uses Bubble Sort or Selection Sort inside.

WORKED EXAMPLE

Worked example: average of an array.

Poor modularity (monoblock)

All logic sits in one block, so you cannot reuse the sum.

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Set total = 0
Step 5: Set avg = 0
Step 6: For I = 1 to n
Step 7:  total = total + A[I]
Step 8: End For
Step 9: avg = total / n
Step 10: Print avg
Step 11: End
```

Good modularity (functions)

Two clear functions: CalculateSum() and FindAverage() .

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Print FindAverage(A, n)
Step 5: End

Function FindAverage(A, n)
  Sum = CalculateSum(A, n)
  Avg = Sum / n
  Return Avg
End Function

Function CalculateSum(A, n)
  Set Total = 0
  For I = 1 to n
    Total = Total + A[I]
  End For
  Return Total
End Function
```

SRP check: `CalculateSum` only adds; `FindAverage` only divides. Loose coupling: you can reuse `CalculateSum` to find a total without computing an average. Abstraction: the main algorithm just calls `FindAverage` and does not see the loop.

The source pseudocode numbers array positions from 1. Python numbers them from 0, so the loop is `for i in range(n)`.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

A = [1, 2, 3, 4, 5]
n = 5
print(find_average(A, n))
```

OUTPUT
3.0

REMEMBER IT

SRP in one line: **one function, one job**. If you cannot name the function with a short verb-phrase (`CalculateSum`, `FindAverage`), it is doing too much.

COMMON MISTAKE

Mistake: copying the 1-based loop `for I = 1 to n: total = total + A[I]` into Python. Then `A[1]` skips the first item and `A[5]` crashes.

Fix: in Python use `for i in range(n)` so the indices are 0, 1, 2, 3, 4.

اردو میں سمجھیں

ماڈیولریٹی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔ جانچ کے تین اصول: واحد ذمہ داری (SRP) یعنی ایک فنکشن ایک کام؛ ڈھیلی جڑت (Loose Coupling) تاکہ ایک حصہ بدلنے سے دوسرے نہ ٹوٹیں؛ اور تجرید (Abstraction) یعنی اندرونی تفصیل چھپا کر صرف کام منگوانا، جیسے سورٹ فنکشن بلائے بغیر یہ جانے کہ ببل ہے یا سلیکشن۔

CHECK YOURSELF

1. What does SRP say each module should do?
2. In the good average algorithm, which function hides the loop?
3. Why is the monoblock harder to reuse?

Answers: 1. One specific task. 2. `CalculateSum` (`FindAverage` only divides; the main program only prints). 3. All logic is in one block, so you cannot call "just the sum" from another algorithm.

